Comparing TurboQuant+ with traditional quantization methods on real tasks

Stanford CS224N {Custom, Default} Project

Matias Soto Carvajal
Faculty of Computer Science
Ruhr-Universität Bochum
name@stanford.edu

Name 2
Faculty of Computer Science
Ruhr-Universität Bochum
name@stanford.edu

Name 3
Faculty of Computer Science
Ruhr-Universität Bochum
name@stanford.edu

Abstract

An abstract should concisely (less than 300 words) motivate the problem, describe your aims, describe your contribution, and highlight your main finding(s).

1 Key Information to include

- Mentor:
- External Collaborators (if you have any):
- Sharing project:

2 Introduction

The introduction explains the problem, why it's difficult, interesting, or important, how and why current methods succeed/fail at the problem, and explains the key ideas of your approach and results. Though an introduction covers similar material as an abstract, the introduction gives more space for motivation, detail, references to existing work, and to capture the reader's interest.

(Quick introduction about KV Cache, memory usage, turboquant approach, and implementation of this.)

On Large Language Models the tokens are computed sequencially during the inference.not lain (2025). Thanks to this behaviour the model is capable of remember things in the inference proccess in order to give more accurate answers and make a conversation with the user.

The Key-Value Cache (KV Cache) works by ... (explain how kv cache works)

The main problem with this approach is that in longer context scenarios (for example, a really long conversation between an AI agent and a user) this cache starts to use a lot of the VRAM of the device, and in the case of small weight models it can be higher that the memory usage of the own model. When this happens, the AI Agent starts to use multiple techniques in order to reduce the usage on memory. These methods can be:

1. Summarize the entire conversation.
2. Drop central values.
3. etc.

In any case, the result is the same: we lose information in favour of more free memory for saving more KV Cache. This can lead to serious problems during the inference, where the model can forget specific things and, by consecuense, start to hallucinate about it previous responses. Multiple approches we're made to reduce the impact of the KV Cache on memory.

The most recent and relevant method on this topic is the TurboQuant method, researched and made by Google. This paper had some importants key finding on their research: finish this im lazy.

3 Related Work

This section helps the reader understand the research context of your work by providing an overview of existing work in the area.

4 Approach

This section details your approach(es) to the problem. For example, this is where you describe the architecture of your neural network(s), and any other key methods or algorithms.

Para experimentar y descubrir si existe algún beneficio real aplicando las cuantizacionea de TurboQuant es necesario tener una base sólida por donde partir. Los beneficios teóricos de TurboQuant se empiezan a mostrar en la compresión de memoria durante un trabajo de largo contexto.

4.1 Dataset

Se decidio utilizar principalmente el dataset LongBench V2 Bai et al. (2025) debido a sus problemas complejos y largos que junto con su metodologia de seleccion multiple permite poner a prueba rapidamente a los LLM y comprobar si la compresion del KV Cache afecta negativamente en su rendimiento.

4.2 Models

Para la ejecucion de los benchmarks se utilizaron los siguientes modelos de peso abierto:

1. Gemma 4 (E4B y E2B) Q8
2. Qwen 3.5 9B Q8
3. Llama 3.1 8B Q8

Todos estos modelos fueron descargados directamente desde HuggingFace. Estos modelos fueron seleccionados ya que son los mas populares dentro de la comunidad de modelos de peso abierto, tienen una buena relacion calidad/tamaño del modelo, y son los mas compatibles con el fork de TurboQuant.

4.3 Framework

Se utilizo el fork Llama-cpp-turboquant. Este fork agrega el codec stack de TurboQuant a Llama.cpp, permitiendo poder cuantizar tanto los pesos como el KV Cache de los modelos. Ademas, al ser un fork directo de Llama.cpp, hereda todo lo bueno de este, como lo es la compatibilidad con distintos backends de inferencia (CUDA, Vulkan, Metal), iniciar servidores de LLM, y la compatibilidad con multiples modelos de distintos provedores.

4.4 Entorno de trabajo

Todas las pruebas fuerron ejecutadas dentro de una instancia rentada de Vast.ai, con la que se tuvo acceso a una RTX 5090 32GB. Gracias a esto se pudo aumentar la muestra de las pruebas para poder tener resultados mas precisos. Desafortunadamente, el fork de Llama.cpp.turboquant no trae consigo binarios prebuildeados de los binarios, por lo que fue necesario buildearlas desde el mismo contenedor para poder utilizar los binarios mas importantes para las pruebas.

5 Experiments

This section contains the following.

5.1 Data

The dataset for the main benchmark of the project is LongBench V2. This dataset is specialized on long context, hard to resolve problems.(Bai et al., 2025). Describe the dataset(s) you are using

(provide references). If it's not already clear, make sure the associated task is clearly described. Being precise about the exact form of the input and output can be very useful for readers attempting to understand your work, especially if you've defined your own task.

5.2 Evaluation method

Describe the evaluation metric(s) you use, plus any other details necessary to understand your evaluation. Some projects will have clear metrics from prior work on given datasets, but we realize that other projects will define their own metrics. If you're defining your own metrics, be clear as to what you're hoping to measure with each evaluation method (whether quantitative or qualitative, automatic or human-defined!), and how it's defined.

5.3 Experimental details

Report how you ran your experiments (e.g., model configurations, learning rate, training time, etc.)

5.4 Results

Report the quantitative results that you have found. Use a table or plot to compare results and compare against baselines.

- If you're a default project team, you should **report the accuracy and Pearson correlation scores you obtained on the test leaderboard** in this section. You can also report dev set results if you'd like.
- Comment on your quantitative results. Are they what you expected? Better than you expected? Worse than you expected? Why do you think that is? What does that tell you about your approach?

6 Analysis

Your report should include *qualitative evaluation*. That is, try to understand your system (e.g., how it works, when it succeeds and when it fails) by inspecting key characteristics or outputs of your model.

7 Conclusion

Summarize the main findings of your project and what you have learned. Highlight your achievements, and note the primary limitations of your work. If you'd like, you can describe avenues for future work.

8 Ethics Statement

What are the ethical challenges and possible societal risks of your project, and what are mitigation strategies?

References

Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2025. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks.

not lain. 2025. Kv caching explained: Optimizing transformer inference efficiency.

A Appendix (optional)

If you wish, you can include an appendix, which should be part of the main PDF, and does not count towards the 6-8 page limit. Appendices can be useful to supply extra details, examples, figures,

results, visualizations, etc. that you couldn't fit into the main paper. However, your grader *does not* have to read your appendix, and you should assume that you will be graded based on the content of the main part of your paper only.